Java Backend Architecture

Interview Series

Chapter 18.10

Monitoring & Debugging Nginx

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.10 – Monitoring & Debugging Nginx

Overview

Effective Nginx monitoring exposes request rates, error rates, upstream latency, and cache hit ratios to ensure Java microservice SLOs are met. Structured JSON access logs, stub_status metrics, and Prometheus exporters provide observability layers. Understanding nginx -t, nginx -s reload, and debug_connection for real-time debugging is essential for incident response.

Key Definitions

Term	Definition
stub_status	Module exposing basic Nginx connection metrics at a configurable URI.
access_log	Per-request log with configurable format. JSON format enables structured log aggregation.
error_log	Server-level error log. Levels: debug, info, notice, warn, error, crit, alert, emerg.
nginx -t	Tests configuration syntax without applying it. Always run before nginx -s reload.
nginx -s reload	Sends SIGHUP to master: graceful config reload, no connection drops.
nginx -T	Dumps complete parsed configuration including all included files.
$request_time	Total time from receiving first byte to sending last byte of response (client-to-client).
$upstream_response_time	Time from Nginx connecting to upstream until last byte of response received.
debug_connection	Enable debug logging for specific client IP without global debug overhead.
open_file_cache	Caches file descriptors and stat results. Reduces syscalls for static file serving.

Diagram 1: Metrics Flow to Grafana

```
Nginx
|-- stub_status --> nginx-prometheus-exporter --> Prometheus --> Grafana
|-- access_log  --> Filebeat/Promtail --> Elasticsearch/Loki --> Grafana
|-- error_log   --> Filebeat/Promtail --> Elasticsearch/Loki --> Alerts

Key dashboards:
- Request rate (req/s) by status code
- Upstream latency (p50/p95/p99 from $upstream_response_time)
- Active connections over time
- Cache hit rate ($upstream_cache_status)
- Error rate (4xx/5xx)
```

Diagram 2: Request Time Breakdown

```
Client           Nginx            Java Backend
  |                |                   |
  |-- TCP connect ->|                  |
  |-- TLS handshake>|                  |
  |-- HTTP request ->|                  |
  |                 |-- TCP connect ---->|   $upstream_connect_time
  |                 |-- HTTP request --->|   $upstream_header_time
  |                 |<-- Response -------|   $upstream_response_time
  |<-- Response ----|                   |
  |                                     |
  $request_time = total client-to-client
  Nginx overhead = $request_time - $upstream_response_time
```

Code Examples

Stub Status + JSON Access Log:

```
http {
    log_format json_log escape=json
      '{'
        '"time":"$time_iso8601",'
        '"remote_addr":"$remote_addr",'
        '"method":"$request_method",'
        '"uri":"$request_uri",'
        '"status":$status,'
        '"bytes":$bytes_sent,'
        '"request_time":$request_time,'
        '"upstream_time":"$upstream_response_time",'
        '"upstream_addr":"$upstream_addr",'
        '"cache":"$upstream_cache_status",'
        '"user_agent":"$http_user_agent"'
      '}';

    access_log /var/log/nginx/access.log json_log buffer=64k flush=5s;
    error_log  /var/log/nginx/error.log  warn;

    server {
        location /nginx_status {
            stub_status;
            allow 127.0.0.1;
            allow 10.0.0.0/8;
            deny all;
        }
    }
}
```

Debug Specific Client Connection:

```
# Debug one client IP without global debug log overhead
events {
    debug_connection 203.0.113.5;  # client IP to debug
}
error_log /var/log/nginx/debug.log debug;

# Check config before reload (always)
$ nginx -t
# Output: nginx: configuration file /etc/nginx/nginx.conf test is successful

# Reload with zero downtime
$ nginx -s reload

# Watch error log for issues
$ tail -f /var/log/nginx/error.log | grep -v 'info'
```

Prometheus Exporter (Docker):

```
# nginx-prometheus-exporter
docker run -p 9113:9113 nginx/nginx-prometheus-exporter:latest   --nginx.scrape-uri=http://nginx:80/nginx_status

# Key Prometheus metrics exposed:
# nginx_connections_active
# nginx_connections_waiting
# nginx_http_requests_total
# nginx_up (1 = healthy, 0 = unreachable)

# Grafana alert: rate(nginx_http_requests_total{status=~"5.."}[5m]) > 10
```

Interview Q&A – Monitoring & Debugging Nginx

Q1. What does stub_status expose and how is it used?

stub_status exposes: Active connections (currently open), server accepts (total TCP accepts), handled (should equal accepts; drops if different), requests (total HTTP requests), Reading (reading client request headers), Writing (sending response to client), Waiting (keepalive idle connections). nginx-prometheus-exporter scrapes stub_status and exposes counters for Prometheus. High Waiting: many keepalive connections (normal). High Writing: slow responses.

Q2. How do $request_time and $upstream_response_time differ?

$request_time: total time from receiving first byte of client request to sending last byte of response, including client network time. $upstream_response_time: time Nginx spent waiting for upstream Java backend (connect + response). Nginx overhead = $request_time - $upstream_response_time. High upstream_response_time: Java backend slow. High gap: slow client network or large response body. Log both for latency attribution.

Q3. Why use buffer and flush in access_log?

access_log /path/log combined buffer=64k flush=5s: writes are buffered in memory (64k) and flushed every 5 seconds instead of per-request fsync. Reduces disk I/O from O(n) per request to O(1) per interval. Critical for high-throughput Nginx (10k+ req/s) where synchronous log writes become the bottleneck. Trade-off: up to 5 seconds of logs lost on crash. Use buffer=off for audit logs that must not be lost.

Q4. How do you debug a specific client connection without enabling global debug log?

debug_connection 203.0.113.5 in events block enables debug logging only for that IP. error_log /var/log/nginx/debug.log debug sets the log level. Without debug_connection, global debug log would generate gigabytes of output per second under load. Use to trace exact request processing: which location matched, upstream selected, cache decision, header processing for the problematic client.

Q5. What does nginx -T show and when is it useful?

nginx -T dumps the complete parsed configuration including all included files (mime.types, conf.d/*.conf). Essential for: verifying the effective configuration after includes, debugging merge of multiple config fragments, checking that ssl_certificate paths are correctly resolved, auditing security settings across a complex configuration hierarchy. Use before production deployments to validate the complete config.

Q6. How do you monitor upstream health from Nginx logs?

Monitor $upstream_addr changes in access logs: if a server disappears from $upstream_addr, it was marked down (max_fails exceeded). Track $upstream_response_time P95 per upstream address. Alert on: error rate (status 502, 504) per upstream, missing upstreams (all requests going to one server), cache bypass rate increase. Pair with Java Actuator metrics for end-to-end latency analysis.

Q7. How do you perform log rotation without dropping access log entries?

Logrotate with postrotate script: nginx -s reopen (sends SIGUSR1) causes Nginx to close old log file descriptors and open new ones. Without reopen, Nginx keeps writing to the rotated file (moved/renamed by logrotate). Logrotate config: daily, compress, postrotate nginx -s reopen. Alternative: syslog-based logging (access_log syslog:server=log) where rotation is handled by syslog daemon.

Q8. How do you find which location block is handling a request?

Use add_header X-Served-By $hostname; add_header X-Cache-Status $upstream_cache_status in locations for runtime visibility. In debug log (debug_connection): 'using configuration' message shows which location matched. nginx -T shows config order. curl -I response headers show custom debug headers. For complex routing bugs, temporarily add access_log per location to identify which location handles specific URIs.

Q9. What metrics should trigger alerts for Java services behind Nginx?

Alert on: error_rate = rate(requests{status=~'5..'}[5m]) / rate(requests[5m]) > 0.01 (1%); p99_latency = histogram_quantile(0.99, upstream_response_time) > SLO_threshold; active_connections approaching worker_connections limit; ssl_certificate_expiry < 30 days; upstream_response_time spike (Java GC pause, DB slowdown). Set PagerDuty for error_rate > 5% and P99 > 2x SLO.

Q10. How do you tune Nginx worker processes and connections for a Java deployment?

worker_processes auto (1 per CPU). worker_connections 4096 (total = cores x 4096). keepalive 32 (upstream pool). proxy_http_version 1.1. open_file_cache max=1000 (static). worker_rlimit_nofile 65535 (match ulimit). access_log buffer=64k flush=5s. Benchmark with wrk or ab: wrk -t4 -c400 -d30s http://nginx/api/endpoint. Monitor worker CPU usage -- if saturated,

increase worker_processes or optimise upstreams.